INFORME TÉCNICO: ACTIVIDAD 7 - PERSISTENCIA EN DISCO Y MANEJO DE ESTRUCTURAS EN C++

Carl Menger
Escuela de Educación Técnica N° 999 "Österreichische Schule der Nationalökonomie"
Laboratorio de Programación (LPR) — 5° Año 3°
Prof. Mansilla Muñoz York
16 de Octubre de 2026

Introducción

El presente informe documenta el desarrollo y verificación experimental del proyecto 'piedrapapelotijera' para la asignatura Laboratorio de Programación. Se analiza la distinción entre la memoria RAM y el almacenamiento secundario en disco mediante la implementación de las librerías <fstream>, estructuras de datos heterogéneas (struct) y el control riguroso de búferes de entrada en C++11.

Desarrollo

Cuestionario Técnico de Control

Pregunta 1: Distinción entre RAM y Almacenamiento en Disco

Respuesta: La memoria RAM es un medio volátil que pierde su contenido al finalizar el proceso.

Al guardar las partidas en 'src/historial_partidas.txt' mediante flujos fstream, la información se escribe físicamente en el disco secundario (no volátil). Por ello, al cerrar la consola y reiniciar el programa, la Opción 2 recupera el historial completo desde el disco.

Pregunta 2: Impacto del Modo ios::app

Respuesta: Si se omite la bandera ios::app al instanciar ofstream archivo("src/historial_partidas.txt"), el archivo se abre por defecto en modo de truncamiento (ios::out). Esto sobrescribiría el archivo en cada nueva partida, borrando todo el registro histórico y conservando únicamente la última jugada realizada.

Pregunta 3: Gestión del Búfer de Entrada con cin.ignore() y limpiarBuffer()

Respuesta: Al leer datos numéricos con cin >> opcion, el carácter de salto de línea ('\n') generado por la tecla Enter permanece en el búfer. Si a continuación se llama a getline(), esta lee el '\n' pendiente como una cadena vacía y omite la entrada del usuario. La función limpiarBuffer() vacía el flujo cin para permitir capturar correctamente el nombre del jugador.

Pregunta 4: Ventajas de Encapsulamiento con struct Partida

Respuesta: La estructura struct Partida permite agrupar variables de tipos heterogéneos (int y string) bajo un mismo tipo de dato definido por el usuario. Esto simplifica el paso de parámetros a funciones mediante referencias constantes (const Partida& p), reduce el riesgo de inconsistencias y mejora la modularidad del código.

Pregunta 5: Transferencia y Aplicación al Proyecto Integrador Anual (P.I.A.)

Respuesta: El esquema de persistencia CSV mediante ofstream e ifstream es directamente aplicable al P.I.A. para registrar eventos de sensores (temperatura, humedad), fichaje de usuarios RFID, auditoría de accesos o almacenamiento de estados de variables críticas en archivos de texto persistentes.

Conclusión

El desarrollo de la Actividad 7 permitió consolidar de forma práctica la diferencia fundamental entre el almacenamiento volátil en memoria RAM y la persistencia no volátil en disco mediante la librería <fstream> de C++11. La implementación de la estructura struct Partida demostró ser una solución técnica eficiente para encapsular variables de distinta naturaleza en una única unidad conceptual, mejorando la modularidad del código y facilitando el paso limpio de parámetros a través de referencias constantes.

Asimismo, la gestión explícita del búfer de entrada mediante cin.ignore() aseguró un control de flujo robusto, evitando errores comunes en la captura de cadenas de texto por consola. En definitiva, la arquitectura modular construida no solo garantiza el resguardo completo del historial de partidas en formato CSV utilizando el modo ios::app, sino que además provee un patrón de diseño reutilizable y directamente transferible para la persistencia de datos y auditoría de eventos en el Proyecto Integrador Anual (P.I.A.).

Referencias

E.E.S.T. N° 999. (2026). Actividad 7: Mini juego 'Piedra, Papel o Tijera' con persistencia de datos (struct + fstream) [Guía de trabajos prácticos]. Laboratorio de Programación.

Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.